
Comprehensive Codebase Audit Report

Legal Tech Application (Atrium / PracticePro)

Implementation Status | Regression Analysis | Net Impact | Logic Gaps

Date: May 13, 2026

Auditor: Z.ai Automated Audit System

Scope: Full-stack React + Convex application (property management focus)

This audit was conducted against the full source repository, covering 150+ source files across the frontend (React/TypeScript), backend (Convex), and supporting infrastructure. The analysis focuses on four dimensions: implementation completeness, regression risk, net impact of recent architectural changes, and logical coherence of the codebase.

Table of Contents

1. Executive Summary	4
2. Implementation Status	4
2.1 Fully Functional Features	4
2.2 Partially Implemented Features	5
2.3 Missing / Dead Features	6
3. Regression Analysis	6
3.1 Modularization Overhaul: Context Architecture	6
3.2 Hard-Coding Remediations: Partial Success	7
3.3 Property Management Regressions	7
4. Net Impact Assessment	7
4.1 What Has Objectively Improved	8
4.2 What Has Objectively Worsened	8
5. Logic Gaps	9
5.1 Critical Logic Gaps	9
5.1.1 CollectRentModal Discards Unit Context	9
5.1.2 PropertyTrackingView Is Orphaned	9
5.1.3 withFirmAuth Does Not Authenticate	9
5.2 Hook Protocol Deviations	10
5.3 Nonsensical Code Blocks	10
6. Security Audit	11
6.1 Exposed API Keys	11
6.2 Convex Authentication Failure	11
6.3 XSS Vectors	11
7. Architecture Deep Dive	12

7.1 The God Object Problem	12
7.2 Property-Finance Integration Gap	12
7.3 Convex Schema Disconnection	12
8. Prioritized Recommendations	12
9. Property Management: Specific Issue Map	14
10. Convex Data Hydration Gaps	15
11. Conclusion	15

1. Executive Summary

This audit examines a comprehensive legal technology application built with React and TypeScript on the frontend, Convex as the serverless backend, and a modular context-based state management architecture. The application serves Nigerian legal practitioners with features spanning matter management, document drafting, research, billing, communications, and a recently added property management module (branded "Atrium"). The property management module is the primary focus of this audit, as it represents the newest and most actively evolving section of the codebase.

The audit identifies 5 critical-severity issues, 8 high-severity issues, 7 medium-severity issues, and several low-severity items. The most significant findings concern: (1) exposed API keys in the client bundle, (2) absent server-side authentication on 50+ Convex functions, (3) orphaned UI components that are imported but never rendered, and (4) dual payment recording systems that are not synchronized. On the positive side, the modularization effort has produced a clean context hierarchy with no circular dependencies, the Convex mutations are comprehensive, and the Atrium Revenue Engine sub-system is fully wired to the backend with real data flows.

Category	Critical	High	Medium	Low
Security	5	1	0	0
Architecture	2	2	3	2
Property Management	2	3	2	3
Code Quality	0	2	2	4

Table 1: Audit Findings Summary by Category and Severity

2. Implementation Status

This section maps which features are fully functional, which are partially implemented, and which are entirely missing. The property management module shows a pattern of rich visual components that are rendered but whose data flows are incomplete or disconnected from persistence layers.

2.1 Fully Functional Features

The following components and features are fully implemented with complete data flows from Convex backend through context providers to the UI, including write-back through mutations:

- ☑ PropertyManagerView: List view with compact/full modes, search, bulk select, delete, and edit. All event handlers are wired and data persists to Convex.

- â─¢ PropertyForm: Comprehensive multi-unit property creation and editing with auto lease-end calculation, amenity management, file upload, unit ripple logic, and caution deposit routing to ledger.
- â─¢ RevenueEngine (Atrium): Fully wired tab container with real Convex queries for ledger entries, service charges, leads pipeline, and automation logs. All sub-components (ServiceChargeMonitor, LedgerManager, VacancyPipeline, AutomationCenter) use real Convex mutations.
- â─¢ LedgerManager: Complete CRUD for ledger entries, filtering by type/status/search, receipt generation via print window. Fully connected to Convex ledger_entries table.
- â─¢ BillingView + InvoiceDetailView: Invoice list with search, filtering, and status management. Invoice preview with zoom, PDF generation, mark-as-paid, revert, and delete.
- â─¢ ReceiptDetailView: Receipt preview with zoom and PDF generation for paid invoices.
- â─¢ ExpandablePropertyGroup: Clean expandable unit list with click, edit, and delete handlers.
- â─¢ RecentPropertiesWidget + PropertyReports: Dashboard widgets and KPI reports with real data.
- â─¢ Auth system: Login, signup, MFA, firm management, invite codes, and account recovery are all fully implemented.

2.2 Partially Implemented Features

These features have UI components rendered but suffer from incomplete data flows, missing handlers, or disconnected persistence. They represent the highest-risk areas for user-facing bugs.

Feature	Component	Status	Impact
Unit-level receipt issuance	CollectRentModal	Broken	ModalContext with unit data is silently discarded by ModalManager
Unit tracking (payments, timeline, maintenance)	PropertyTrackingView	Orphaned	Component is imported but never rendered in PropertyDetailView
Maintenance logging	PropertyDetailView cog menu	Stub	Only toggles unit status to "Maintenance"; no MaintenanceRecord created
Bulk edit properties	BulkEditPropertyModal	Mismatched	Status/category dropdowns use different enum values than the rest of the app
Payment gateway	PaymentGatewayModal	Simulated	setTimeout-based simulation; no real payment processing occurs
Issue Receipt (quick action)	PropertyDetailView	Partial	Quick action works for single-tenant properties but ignores unit context for multi-unit

Table 2: Partially Implemented Features

2.3 Missing / Dead Features

The following features are either entirely absent from the codebase or exist as dead code that is never executed. The most significant gap is the `PropertyTrackingView` component, which contains a fully built timeline, rent history table, maintenance Kanban board, and lease management UI, but is imported in `PropertyDetailView` without ever being rendered. This means the entire unit-level tracking experience that users expect is invisible.

- â■ PropertyTrackingView: Imported at line 18 of `PropertyDetailView.tsx` but has zero JSX references. Contains complete timeline, rent history, maintenance Kanban, lease setup, and receipt download functionality. All dead code.
- â■ Valuations workflow: No dedicated valuations dialogue exists. The word "valuation" appears only as a simple number input field. There is no valuation history, professional valuation report, or date tracking.
- â■ Unit-level edit in detail view: Unit cards in the Units tab have no "Edit" button. The only edit access is through `ExpandablePropertyGroup` in the sidebar list view, which most users will not discover.
- â■ 7 unimplemented context handlers: `handleBulkArchiveTasks`, `handleArchiveAllDoneTasks`, `handleUpdateWorkflow`, `handleUpdateChecklistItem`, `handleRenamePage`, `handleUpdatePageContent`, `handleDeleteNotebook` are declared in context types but have no implementation in any hook. Components calling these will crash with `TypeError`.

3. Regression Analysis

This section examines whether recent architectural changes, including the modularization overhaul and hard-coding remediations, have introduced new bottlenecks, bugs, or behavioral regressions. The analysis is based on static code analysis of data flows, event handler wiring, and state management patterns.

3.1 Modularization Overhaul: Context Architecture

The transition to a multi-context architecture (12 providers in `AppProvider`) has produced a clean hierarchy with no circular dependencies. However, it introduced a significant performance regression through the "God Object" `AppState` pattern. The `DataProvider` holds a single monolithic `AppState` object with 35+ array fields, and every domain context (Core, Matter, Finance, Execution, Document) is just a view/projection of this same object. The `contextActions` object in `DataProvider` (line 127) is typed as "any" and is NOT memoized, meaning it is recreated on every render. Since `DataActionsContext` provides this object, every consumer of `useDataActions()` re-renders on any state change, regardless of which slice of state they actually consume. This is the number one performance bottleneck in the architecture.

Additionally, the modularization has created a dual-exposure problem where the same data arrays (matters, contacts, invoices, tasks, events, documents, etc.) are exposed through both CoreContext and their respective domain contexts simultaneously. While this does not cause data inconsistency (both read from the same appState), it creates architectural confusion and potential for stale reads if only one context is updated in a future refactor.

3.2 Hard-Coding Remediations: Partial Success

The codebase shows evidence of hard-coding remediation efforts, particularly in the tier and pricing system (src/constants/tiers.ts) which is properly centralized as a single source of truth. However, several hard-coded values remain that represent regression risks:

- â■ Production Convex URL: src/main.tsx line 14 falls back to "https://gregarious-malamute-537.convex.cloud" if the VITE_CONVEX_URL environment variable is missing. This should fail loudly instead of silently connecting to production.
- â■ Demo firm IDs: "demo-firm-id" and "atrium-demo-firm-id" are hardcoded in demoData.ts. The demo bypass in AuthContext.tsx (if token === "demo@practicepro.ng") could be exploited if not properly gated.
- â■ Duplicate EMPTY_APP_STATE: Two separate definitions exist in types.ts and mockData.ts, imported by different files. If these diverge, subtle initialization bugs will occur.
- â■ Management fee default: CollectRentModal defaults to 2.5% management fee when property.managementFeePercentage is not set. This should be a configurable firm-level setting, not a hardcoded fallback.

3.3 Property Management Regressions

The property management module shows clear signs of iterative development where new features were added without fully integrating them with the existing architecture. The most concerning regression is the dual payment recording system:

Dual Payment Systems (Not Synchronized): "Record Rent Payment" (from unit cog menu) writes to the Convex ledger_entries table via the RecordRentPaymentModalWrapper. "Issue Receipt" (CollectRentModal) writes to property.rentPaymentHistory and generates a management fee invoice. These two systems are completely separate data stores with no synchronization. A payment recorded via "Record Rent Payment" will not appear in the rent history table, and a receipt issued via CollectRentModal will not appear in the ledger. This creates an accounting discrepancy that will compound over time and make financial reporting unreliable.

Another regression is the ReceiptDetailView showing the client (landlord) name instead of the tenant name in the "Received From" field for property rent receipts. This is a legal compliance issue in jurisdictions like Lagos where receipts must accurately identify the payer.

4. Net Impact Assessment

This section objectively evaluates what has improved and what has worsened as a result of recent architectural changes, security hardening efforts, and feature additions.

4.1 What Has Objectively Improved

Area	Improvement	Evidence
Context Architecture	Clean provider hierarchy with no circular dependencies; 12 well-separated contexts	All context dependencies flow strictly top-down; verified across all 14 context files
Convex Backend Coverage	All mutations used by frontend exist in backend; comprehensive CRUD operations	20+ mutations verified with matching frontend consumers; cascade deletes implemented
Tier/Pricing Centralization	Single source of truth in constants/tiers.ts for all pricing, feature flags, and limits	No scattered hard-coded pricing found outside this module
Sanitization Module	Well-implemented DOMPurify-based sanitization used in 9+ components	Proper allowlist configuration; imported in SafeHtml.tsx and markdownUtils.ts
Atrium Revenue Engine	Fully wired property financial sub-system with real Convex data flows	All 4 sub-components use real mutations; no mock data; KPIs computed from actual ledger entries
Error Boundaries	GlobalErrorBoundary and ConvexErrorBoundary provide graceful failure handling	Wraps main App component; catches both React and Convex errors

Table 3: Verified Improvements

4.2 What Has Objectively Worsened

Area	Regression	Evidence
Server-Side Auth	withFirmAuth does not verify caller identity; 50+ functions unauthenticated	Only authHelpers.ts calls getUserIdentity(); all others trust client-provided firmId
API Key Exposure	2 Gemini API keys hardcoded in source; visible in browser bundle	aiUtils.ts, geminiService.ts, AloaChat.tsx contain live keys in plain text
Render Performance	Non-memoized contextActions in DataProvider causes cascade re-renders	contextActions typed as "any"; recreated every render; all consumers re-subscribe
Property Data Integrity	Dual payment systems (ledger vs rentPaymentHistory) not synchronized	CollectRentModal writes to property.rentPaymentHistory; RecordRentPayment writes to ledger_entries

Dead Code Volume	PropertyTrackingView fully implemented but never rendered; 7 unimplemented handlers	Import exists but zero JSX references; handlers declared in types but no hook provides them
Type Safety	100+ "as any" assertions mask runtime type mismatches	Concentrated in PropertyDetailView (12+), DataProvider (10+), demoData.ts (20+)

Table 4: Verified Regressions

5. Logic Gaps

This section identifies code blocks that are nonsensical, incomplete, or deviate from the established hook protocol. These represent the most actionable items for immediate remediation, as they directly cause user-facing bugs or data integrity issues.

5.1 Critical Logic Gaps

5.1.1 CollectRentModal Discards Unit Context

When a user clicks "Issue Receipt" from a unit cog menu in PropertyDetailView (line 819), the code correctly passes modalContext containing { unitName, tenantName, rentAmount } to the openModal call. However, ModalManager (lines 323-330) completely discards this modalContext and passes only the parent property object to CollectRentModal. This means the modal always shows the property-level tenant and rent amount, not the specific unit that was clicked. For multi-unit properties, this is a critical bug: users will issue receipts with incorrect tenant names and incorrect rent amounts. The fix is to pass modalContext as a prop to CollectRentModal and use unit-specific data when available, following the same pattern already implemented in RecordRentPaymentModalWrapper (lines 138-148).

5.1.2 PropertyTrackingView Is Orphaned

PropertyTrackingView is imported at line 18 of PropertyDetailView.tsx but is never rendered in any JSX. This component contains a fully implemented timeline, rent history table, maintenance Kanban board with four status columns, lease setup form, manual event addition, and receipt download functionality. All of this is dead code. The "Add Payment" and "Save Record" flow that the user reported as not working is part of this orphaned component. When users navigate to a unit and try to add a payment, they are actually interacting with inline UI in PropertyDetailView that only calls handleSave locally, and the onUpdate callback is never connected to a Convex mutation. The fix is to integrate PropertyTrackingView into the PropertyDetailView units tab, replacing the current inline stubs with this fully-featured component.

5.1.3 withFirmAuth Does Not Authenticate

The withFirmAuth function in convex/lib/withAuth.ts (lines 15-38) is supposed to be the authentication middleware for Convex functions, but it has three critical flaws: (1) it never calls

ctx.auth.getUserIdentity() to verify who the caller is, (2) it proceeds with the handler even when no matching user is found (the comment says "Allow the query to proceed in read-only mode"), and (3) it uses an inefficient full-table scan (take(500)) instead of an indexed lookup. This means that any client knowing a firmId can read, create, update, or delete any firm's data. Critical functions like purgeFirmData, adminDeleteUser, and adminForceVerify have no role verification whatsoever. This is a severe security vulnerability that must be remediated before any production deployment.

5.2 Hook Protocol Deviations

The established hook protocol requires each hook to accept (appState, actions) and return handler functions with try/catch error handling and toast notifications. Several hooks deviate from this protocol:

Hook	Deviation	Risk
useMatters	appState parameter typed as "any"; loses type safety for the largest hook	High
useMessaging	No error handling for failed messages; silent failures	Medium
useTasks	No error handling for bulk operations; missing handleBulkArchiveTasks and handleArchiveAllDoneTasks implementations	High
useResearch	No error handling; crashes on API failures	Medium
useCommunications	Different signature (takes only actions, not appState); inconsistent with protocol	Low

Table 5: Hook Protocol Deviations

5.3 Nonsensical Code Blocks

Several code blocks exhibit logic that is either self-contradictory, unreachable, or produces incorrect results:

- ❗ MFA Debug Log in Production: Login.tsx line 110 logs the actual MFA verification code to the browser console in production builds. This is a security vulnerability that allows anyone with access to the browser dev tools to bypass MFA.
- ❗ Hardcoded appMode: AuthContext always provides appMode: AppMode.Multi (line 488). The AppMode enum has Solo and Multi values, but there is no mechanism to ever set Solo mode. App.tsx has conditional logic for AppMode.Solo (line 499) that is unreachable dead code.
- ❗ Dead State Fields: DataStateContext provides isSaving (always false), isOutdated (always false), and availableBackups (always []) as part of its public API. These fields mislead consumers into thinking functionality exists when it does not.

â■ BulkEditPropertyModal Enum Mismatch: The modal offers status values (Available, Leased, For Sale, Maintenance, Under Renovation) that do not match the PropertyStatus enum used throughout the app (Occupied, Vacant, Listed). The category options also differ. This means bulk edits will set values that the app's filtering and display logic cannot recognize.

â■ Receipt Shows Landlord Name Instead of Tenant: ReceiptDetailView uses client?.name for the "Received From" field, but for property rent receipts, the payer is the tenant, not the landlord. This produces legally incorrect receipts.

â■ Math.random() for Invoice/Receipt Numbers: Both useProperties.ts and invoiceService.ts use Math.random() to generate receipt and invoice numbers. In concurrent environments, this creates collision risks that could result in duplicate financial documents.

6. Security Audit

The security audit reveals critical vulnerabilities that must be addressed before any production deployment. The most severe findings are exposed API keys and absent server-side authentication.

6.1 Exposed API Keys

Two live Google Gemini API keys are embedded directly in the source code and will be fully visible in the browser bundle to anyone who inspects the deployed application. The keys are AIzaSyAjPumBNTGi8Lzg457yKm4dD0jFAzefXo0 (found in aiUtils.ts, geminiService.ts, and AloaChat.tsx) and AIzaSyDd5ib2A1562gO2PY1FQEISVzwyIaeBAN8 (found in geminiService.ts). Additionally, API keys are passed in URL query strings (aiUtils.ts line 235, GeminiStructurer.ts line 650), which are logged by proxies, CDNs, and browser history. The architectural issue is that AI calls are made directly from the client-side; they should be proxied through Convex actions which execute server-side, keeping the API keys secret.

6.2 Convex Authentication Failure

Only one Convex file (authHelpers.ts line 13) calls ctx.auth.getUserIdentity() to verify the caller's identity. The other 50+ exported Convex functions have no identity verification. They rely solely on firmId being passed as an argument from the client. This means any client that knows a firmId can read, create, update, or delete any firm's data. The most dangerous unauthenticated functions include getFirmData (returns all firm data), dumpAll (exposes entire database), createItem/updateItem/deleteItem (modify any firm's data), purgeFirmData (wipes entire firm), and adminDeleteUser/adminForceVerify (no admin role check). The withFirmAuth middleware that should protect these functions does not actually verify identity and proceeds even when authentication fails.

6.3 XSS Vectors

Thirteen uses of dangerouslySetInnerHTML were found. While most properly pipe through the sanitize() function, four components use parseAloaMarkdown() without sanitization: NoteDetails.tsx

(line 34), MiniAloa.tsx (line 380), AloaChat.tsx (line 1027), and MessagingView.tsx (lines 270, 859). If any of this content comes from AI output or other users, it represents an XSS attack vector.

7. Architecture Deep Dive

7.1 The God Object Problem

The `DataProvider` holds a single monolithic `AppState` object with 35+ array fields. Every domain context (Core, Matter, Finance, Execution, Document) is merely a view or projection of this same `AppState`. The consequence is that any mutation to any field triggers a re-render of `DataProvider`, which re-creates the `contextActions` object. Since this object is typed as "any" and is not memoized with `useMemo`, every consumer of `useDataActions()` re-renders regardless of which slice of state they actually consume. For a legal tech application that may handle hundreds of matters and thousands of documents, this will cause noticeable UI lag as the dataset grows. The fix is to memoize `contextActions` keyed on `baseActions` and all hook return values, and ideally to split `AppState` into domain-specific slices with independent update paths.

7.2 Property-Finance Integration Gap

There is a fundamental architectural gap between the property management module and the finance context. The `FinanceContext` provides invoices, `timeEntries`, and expenses, but these are legal billing constructs. Property financial data is modeled entirely differently: `rentPaymentHistory` is embedded in `Property` objects, `maintenanceHistory` costs are embedded in `Property` objects, and `ledgerEntries` and `serviceCharges` live in `CoreContext` but not `FinanceContext`. There is no `PropertyFinanceContext` or equivalent. This means property earnings are not aggregated into any finance context, property expenses are not included in `FinanceContext.expenses`, and the `useFinance` hook has zero property financial logic. The `RevenueEngine` component must manually traverse `coreState.properties` to compute revenue, occupancy rates, and arrears. A proper integration layer is needed.

7.3 Convex Schema Disconnection

The Convex schema has dedicated tables for property-specific data that are never fetched into `appState`. The `tenancies`, `ledger_entries`, `service_charges`, `leads_pipeline`, and `automation_logs` tables exist in the schema but the `DataProvider`'s `getFirmData` query does not hydrate them into the client state. The Atrium components work around this by calling Convex queries directly (via `useConvex`), but this creates an inconsistency where some data flows through the context system and some bypasses it entirely. This dual-path architecture will become increasingly difficult to maintain as the property module grows.

8. Prioritized Recommendations

The following recommendations are ordered by priority, with the most critical items first. Each item includes a brief description of the issue, the recommended fix, and the estimated impact of the fix.

#	Issue	Severity	Recommended Fix	Impact
1	Exposed Gemini API keys in client bundle	Critical	Remove keys immediately; rotate compromised keys; proxy all AI calls through Convex actions	Prevents unauthorized API usage and cost leakage
2	50+ Convex functions without authentication	Critical	Add <code>ctx.auth.getUserIdentity()</code> to all functions; fix with <code>FirmAuth</code> to block on auth failure	Prevents unauthorized data access and modification
3	CollectRentModal discards unit context	Critical	Pass <code>modalContext</code> prop to CollectRentModal; use unit-specific data when available	Fixes incorrect receipts for multi-unit properties
4	PropertyTrackingView is orphaned	Critical	Integrate PropertyTrackingView into PropertyDetailView units tab; wire <code>onUpdate</code> to <code>updateItem</code>	Restores timeline, maintenance, and payment tracking
5	Dual payment systems not synchronized	Critical	Unify on a single data store; write to both <code>ledger_entries</code> and <code>rentPaymentHistory</code> atomically	Ensures financial data consistency
6	7 unimplemented context handlers	High	Implement in <code>useTasks</code> , <code>useResearch</code> , and document hooks; or remove from context types	Prevents runtime crashes in TasksView, NotesView, etc.
7	MFA debug code in production console	High	Gate behind <code>import.meta.env.DEV</code> ; remove from production builds	Prevents MFA bypass via browser dev tools
8	XSS vectors in 4 components	High	Wrap <code>parseAloaMarkdown()</code> calls with <code>sanitize()</code> in NoteDetails, MiniAloa, AloaChat, MessagingView	Closes XSS attack surface from user/AI content
9	contextActions not memoized	High	Wrap in <code>useMemo</code> keyed on <code>baseActions</code> + all hook returns	Eliminates cascade re-renders; major perf improvement
10	No unit-level edit button	High	Add per-unit edit button in Units tab below the main Edit button; make scrollable for many units	Users can edit individual units without navigating away
11	BulkEditPropertyModal enum mismatch	Medium	Align status/category dropdowns with PropertyStatus and category enums used elsewhere	Prevents setting unrecognized values
12	100+ "as any" type assertions	Medium	Incrementally replace with proper type guards and generics	Restores type safety; catches bugs at compile time

Table 6: Prioritized Recommendations

9. Property Management: Specific Issue Map

This section maps the specific issues raised in the user brief to their root causes in the codebase, providing a direct line from reported symptom to code-level fix.

Reported Issue	Root Cause	File(s)	Fix
Valuations dialogue when property not for sale	Valuation field is a simple number input shown for all non-rental properties; no conditional logic	PropertyForm.tsx:560, PropertyDetailView.tsx:504	Remove valuation field for non-sale properties or add "For Sale" gating condition
Add Payment / Save Record does nothing	PropertyTrackingView (which handles this flow) is imported but never rendered; inline UI has no persistence	PropertyDetailView.tsx:18, PropertyTrackingView.tsx:213-294	Render PropertyTrackingView in Units tab; wire onUpdate to Convex mutation
Activity Timeline does nothing	Same as above; timeline is part of orphaned PropertyTrackingView	PropertyTrackingView.tsx:54, 234-289	Integrate PropertyTrackingView; timeline auto-populates from rent/maintenance events
Maintenance does nothing	Cog menu "Log Maintenance" only toggles status; no MaintenanceRecord created	PropertyDetailView.tsx:845,879	Replace stub with form to create MaintenanceRecord with issue, priority, vendor
Issue Receipt does not work for property owner	CollectRentModal discards unit context; receipt shows landlord instead of tenant	ModalManager.tsx:323-330, ReceiptDetailView.tsx:40-46	Pass modalContext to CollectRentModal; fix ReceiptDetailView to show tenant name
Receipt should be auto-calculated	Rent amount not pre-populated from unit data; user must enter manually	CollectRentModal.tsx:86-95	Pre-fill amount from unit.rentAmount or property.rentalDetails.rentAmount
Receipt should be in units not larger property	ModalManager drops unit-level context; no unit-specific receipt path exists	ModalManager.tsx:323-330	Create unit-level receipt flow; aggregate unit receipts for property-level reporting

Issue Receipt should be in Quick Actions	Quick Actions only appear for non-leased, single-unit properties; multi-unit excluded	PropertyDetailView.tsx:585-635	Add Quick Actions for multi-unit properties; include "Issue Receipt" per unit
Edit button opens property selector	Only when owner contact is deleted/missing; by design but confusing	ModalManager.tsx:372-440	Add per-unit edit button below main Edit; place beside units list
Management fee disbursement not supported	CollectRentModal only generates fee invoice; no disbursement workflow	CollectRentModal.tsx:133-140	Add disbursement tracking: owner collects rent, disburses fee to manager; or vice versa

Table 7: User-Reported Issues Mapped to Root Causes

10. Convex Data Hydration Gaps

The following Convex tables exist in the schema but are never fetched into the client-side `appState` through the standard `DataProvider` pipeline. The Atrium components work around this by calling Convex queries directly, but this creates an architectural inconsistency where some data flows through the context system and some bypasses it entirely.

Convex Table	In Schema	In appState	Workaround
tenancies	Yes	No	Not used by any frontend component
ledger_entries	Yes	Partial	Atrium components query directly via <code>useConvex()</code>
service_charges	Yes	Partial	<code>ServiceChargeMonitor</code> queries directly
leads_pipeline	Yes	Partial	<code>VacancyPipeline</code> queries directly
automation_logs	Yes	Partial	<code>AutomationCenter</code> queries directly
atrium_inbound_messages	Yes	No	Not fetched by any component

Table 8: Convex Tables Not Hydrated in `appState`

11. Conclusion

The codebase demonstrates a sophisticated legal technology application with a well-structured frontend architecture and comprehensive backend coverage. The modularization effort has produced clean

separation of concerns with no circular dependencies, and the Atrium property management sub-system shows real promise with its fully wired Revenue Engine and Convex-integrated components.

However, the audit reveals several critical issues that must be addressed before the property management module can be considered production-ready. The most urgent are: (1) the exposed API keys and absent server-side authentication, which represent severe security vulnerabilities; (2) the orphaned PropertyTrackingView component, which means the entire unit-level tracking experience is non-functional; and (3) the dual payment recording systems, which will produce inconsistent financial data. These issues are not edge cases; they affect core user workflows and data integrity.

The recommended approach is to address the security issues immediately (items 1-2 in the recommendations table), then integrate PropertyTrackingView into the active UI and unify the payment recording systems (items 3-5). The remaining high and medium severity items should be tackled in the next sprint cycle. With these fixes in place, the property management module will have a solid foundation for continued development, and the architecture will support the kind of efficiency gains the user is seeking rather than the confusion currently experienced.